

Introduction to SpringBoot Framework

Introduction to the SpringBoot Application:

- Spring Boot is a module of the spring framework that speeds up the development time.
- It provides an easy and faster way to set up, configure, and run the web-based application.
- Spring Framework along with the embadded tomcat server and lesser configuration is known as Spring Boot Application.

Advantages of Spring Boot

- **Simplified Dependency Management:**

- **Traditional Spring:** You need to manually configure dependencies in pom.xml or build.gradle, often leading to version conflicts or missing dependencies.
- **Spring Boot:** Uses starter dependencies (e.g., spring-boot-starter-web, spring-boot-starter-data-jpa) that bundle all required dependencies for specific functionality, reducing configuration overhead

- **Auto-Configuration:**

- **Traditional Spring:** Requires explicit configuration for beans, data sources, servlets, etc., using XML or Java-based configuration.
- **Spring Boot:** Automatically configures beans and other components based on the dependencies present in the classpath.

Advantages of Spring Boot

- **Embedded Servers:**

- **Traditional Spring:** Requires deploying the application to an external server (e.g., Tomcat, Jetty).
- **Spring Boot:** Comes with embedded servers (e.g., Tomcat, Jetty, Undertow), allowing you to run the application as a standalone JAR file without needing an external server.

- **No XML Configuration:**

- **Traditional Spring:** Often relies on XML configuration files (e.g., applicationContext.xml).
- **Spring Boot:** Encourages Java-based configuration and annotations, eliminating the need for XML files

Advantages of Spring Boot

- **Faster Development:**

- **Traditional Spring:** Requires significant boilerplate code and configuration.
- **Spring Boot:** Reduces boilerplate code and provides a faster development cycle with features like Live Reload and Spring DevTools.

Purpose of Spring Boot

- The primary purpose of Spring Boot is to simplify and accelerate the development of Spring-based applications. It achieves this by:
- Providing default configurations for common use cases.
- Reducing the need for boilerplate code and manual configuration.
- Enabling developers to focus on business logic rather than infrastructure setup.
- Supporting microservices architecture and cloud-native development.

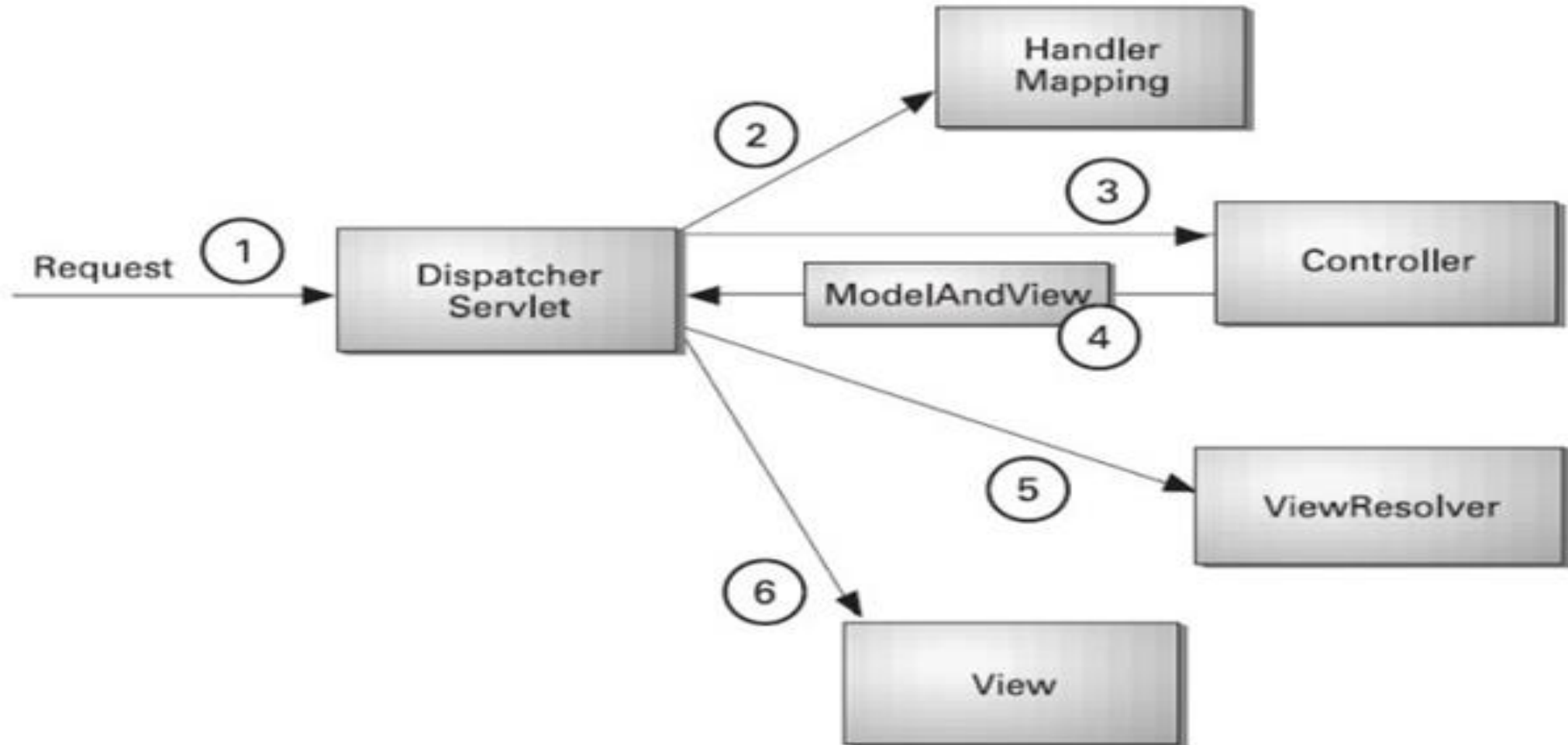
Spring MVC:

- A Spring MVC is a Java framework which is used to build web applications.
- It follows the Model-View-Controller design pattern.
 - **Model**: Represents the data and business logic.
 - **View**: Represents the UI (e.g., HTML, JSP, Thymeleaf).
 - **Controller**: Handles user requests, processes them, and returns the appropriate response.
- It implements all the basic features of a core spring framework like Inversion of Control, and Dependency Injection.
- A Spring MVC provides an elegant solution to use MVC in spring framework by the help of **DispatcherServlet**.
- Here, **DispatcherServlet** is a class that receives the incoming request and maps it to the right resource such as controllers, models, and views.

Understanding the flow of Spring Web MVC

- As displayed in the figure, all the incoming request is intercepted by the DispatcherServlet which works as the front controller.
- The DispatcherServlet gets an entry of handler mapping from the XML file and forwards the request to the controller.
- The controller returns an object of Model And View.
- The DispatcherServlet checks the entry of the view resolver in the XML file and invokes the specified view component.

Understanding the flow of Spring Web MVC



Understanding the flow of Spring Web MVC

- Load the spring jar files or add dependencies in the case of Maven
- Create the controller class
- Provide the entry of controller in the web.xml file
- Define the bean in the separate XML file
- Display the message in the JSP page
- Start the server and deploy the project

Rest

- **REST or Representational State Transfer** is an architectural style that can be applied to web services to create and enhance properties like performance, scalability, and modifiability.
- RESTful web services are generally highly scalable, light, and maintainable and are used to create APIs for web-based applications.

Principles of RESTful applications:

- 1.URI Resource Identification:** A RESTful web service should have a set of resources that can be used to select targets of interactions with clients. These resources can be identified by URI (Uniform Resource Identifiers). The URIs provide a global addressing space and help with service discovery.
- 2.Uniform Interface:** Resources should have a uniform or fixed set of operations, such as PUT, GET, POST, and DELETE operations. This is a key principle that differentiates between a REST web service and a non-REST web service.

Principles of RESTful applications:

3. Self-Descriptive Messages: As resources are decoupled from their representation, content can be accessed through a large number of formats like HTML, PDF, JPEG, XML, plain text, JSON, etc.

4. Use of Hyperlinks for State Interactions: In REST, interactions with a resource are stateless, that is, request messages are self-contained.

- So, the explicit state transfer concept is used to provide stateful interactions.
- URI rewriting, cookies, and form fields can be used to implement the exchange of state. A state can also be embedded in response messages and can be used to point to valid future states of interaction.

Spring AOP:

- Aspect-oriented programming entails breaking down program logic into distinct parts, which are called **so-called concerns**.
- The functions that span multiple points of an application are called cross-cutting concerns.
- These **cross-cutting concerns** are conceptually separate from the application's business logic.
- There are various common good examples of aspects such as logging, auditing, declarative transactions, security etc.
- The key unit of modularity in OOP is the class, whereas in AOP the unit of modularity is the aspect.
- Dependency Injection helps you decouple your application objects from each other, while AOP helps you decouple cross-cutting concerns from the objects that they affect.

Spring AOP:

Important Terminology:

- **Aspect:**
 - A module that has a set of APIs providing cross-cutting requirements.
 - For example, a logging module would be called AOP aspect for logging.
 - An application can have any number of aspects depending on the requirement.
- **Joint Point:**
 - This represents a point in your application where you can plug-in AOP aspect.
 - You can also say, it is the actual place in the application where an action will be taken using the Spring AOP framework.
- **Advice:**
 - This is the actual action to be taken either before or after the method execution.
 - This is the actual piece of code that is invoked during program execution by Spring AOP framework.

Spring AOP:

Important Terminology:

- **Pointcut:**
 - This is a set of one or more joinpoints where an advice should be executed.
 - You can specify PointCuts using expressions or patterns as we will see in our AOP examples.